VERIFICATION REPORT · LEAN 4 / MATHLIB

Strong NP-hardness of asymmetric weighted earliness–tardiness scheduling: a machine-checked verification of Part I

N. G. Hall, H. Kellerer, M. Song, *Asymmetric Weighted Earliness–Tardiness: Scheduling with a Nonrestrictive Common Due Date*, arXiv:2609.16046v1, Part I (Sections 4–9).

Formal development: `AWET_PartI.lean` (one file, 5,002 lines, 336 theorems)
Lean 4 v4.35.0 · Mathlib v4.35.0 · October 2026

Verdict The proof in Part I is **correct**. Every step of the reduction from Restricted Exact Cover by 3-Sets (RXC3) to AWET with the boundary due date $d = \sum_j p_j$ (Sections 6–9) has been formalized in Lean 4 and checked by Lean's kernel. We found no mathematical error; the minor remarks in Section 4 do not affect the result.

- **Formal result** (`AWET.AWET_strong_reduction`). For *every* RXC3 instance X: X has an exact cover $\iff$ the AWET instance built by the paper's Section 8 has a schedule (arbitrary real start times) of cost at most the paper's bound $B = \lfloor A + \Gamma \rfloor$. The instance has $8n + 1$ jobs, and every number in it is at most $(n + 1)^{650}$.
- **Check status.** No `sorry`, no added axioms (only Lean's standard `propext`, `Classical.choice`, `Quot.sound`), no warnings; the compiled module is re-checked declaration by declaration by `leanchecker`.
- **Not machine-checked** (taken as given, as in the paper): NP-completeness of RXC3 (Gonzalez, 1985) and the routine fact that the explicit formulas of the construction can be evaluated in polynomial time.

Paper	Hall, Kellerer, Song, arXiv:2609.16046v1, Part I "Hardness" (paper pp. 18–63)
Claim checked	Theorem I.2: the decision version of AWET with $d = \sum_j p_j$ is strongly NP-complete, in particular unary NP-hard
Coverage	Sections 6–9 in full: canonical reformulation (Lemma I.3, Proposition I.4, membership in NP), Reduction I (Lemmas I.7–I.9), Reduction II (Lemmas I.11–I.18), bounds (Lemma I.19); the worked examples of Appendix B
Formal artifact	`AWET_PartI.lean`: one self-contained file, 5,002 lines, 336 theorems, 117 definitions; Lean 4 v4.35.0 with Mathlib v4.35.0; compiles in about one minute
Issues in the paper	none that affect correctness; seven minor remarks (Section 4)

Contents

1 What exactly was verified **2**
1.1 The claim and its logical shape . . . 2
1.2 The main theorem . . . 3
1.3 The formal problem statement . . . 3
1.4 What the check rests on . . . 5

2 Architecture of the formal proof **5**

3 The proof, step by step **5**
3.1 Step 1 — Canonical schedules realize $\Phi(E)$ (Proposition I.4) . . . 6
3.2 Step 2 — Every schedule costs at least $\min_E \Phi(E)$ (Lemma I.3, (6.1)) . . . 7
3.3 Step 3 — Reduction I: exact cover $\Longleftrightarrow$ prefix exactness (Lemmas I.7–I.9) . . . 9
3.4 Step 4 — The construction of Section 8.1 . . . 9
3.5 Step 5 — Two zero-load states (Lemma I.11 and (8.4)) . . . 12
3.6 Step 6 — Kernel field and scale separation (Lemmas I.12 and I.14) . . . 12
3.7 Step 7 — Exact pair compression and the profile decomposition (Lemma I.13, (8.9)–(8.12)) . . . 13
3.8 Step 8 — The objective with an early anchor and the load lock (Lemma I.15, (8.23)–(8.24)) . . . 15
3.9 Step 9 — Gates 1 and 2: the anchor is early, every block is coded ((8.25), Lemma I.15) . . . 16
3.10 Step 10 — Coded expansion and the master identity (Lemma I.16, (8.26)–(8.31)) . . . 17
3.11 Step 11 — Residual budget and decision gap ((8.19), (8.32)–(8.34)) . . . 18
3.12 Step 12 — Both directions (Lemmas I.17 and I.18) . . . 18
3.13 Step 13 — Polynomial and unary bounds (Lemma I.19) . . . 20
3.14 Step 14 — The worked examples of Appendix B . . . 21

4 Remarks on the paper **23**

5 Reproducing the check **24**

A Map from the paper to the Lean file **24**

B Layout of the Lean file **25**

1 What exactly was verified

1.1 The claim and its logical shape

Theorem I.2 states that the decision version of AWET (one machine; positive integer processing times p_j, earliness weights w_j^E and tardiness weights w_j^L; common due date $d = \sum_j p_j$; question: is there a schedule of cost at most B?) is strongly NP-complete. In the standard sense of Garey and Johnson, the paper proves strong NP-hardness with a transformation $X \mapsto (\mathrm{inst}(X), B(X))$ from RXC3, a problem without numbers, such that

(a) *correctness*: X is a yes-instance $\iff$ $(\mathrm{inst}(X), B(X))$ is a yes-instance;
(b) *polynomially bounded numbers*: every number of $(\mathrm{inst}(X), B(X))$ is at most a fixed polynomial in the size of X;
(c) *polynomial time*: the transformation can be computed in polynomial time.

Membership in NP rests on the canonical set formulation (Lemma I.3, Proposition I.4): a yes-instance has an early set E with $\Phi(E) \le B$, and $\Phi(E)$ is computable in polynomial time.

The Lean development proves (a) for every RXC3 instance, proves (b) with the paper's own polynomial $(n+1)^{650}$, and proves the canonical formulation used for NP membership. Item (c) is not formalized: every constructed quantity is an explicit closed formula (sums over $O(n^2)$ job pairs, roundings, a floor and a ceiling of rational numbers), and its polynomial-time computability is routine. The NP-completeness of RXC3 is taken from the literature, exactly as in the paper.

1.2 The main theorem

The final theorem of the file is the formal core of Theorem I.2.

```
/-- **Theorem I.2 (formal core).** The paper's transformation `X ↦ (X.inst, X.Bnd)` from
Restricted Exact Cover by 3-Sets to AWET with the boundary due date `d = Σ p_j` is correct, and
for every nonempty source instance it uses `8n+1` jobs whose data, due date and threshold are
bounded by the fixed polynomial `(n+1)^650`. -/
theorem AWET_strong_reduction (X : RXC3) :
    (X.HasExactCover ↔ X.inst.Yes X.Bnd) ∧
    (0 < X.n →
      Fintype.card (Job X.n) = 8 * X.n + 1 ∧
      (∀ j, 0 < X.inst.p j ∧ X.inst.p j ≤ ((X.n : ℤ) + 1) ^ 650) ∧
      (∀ j, 0 < X.inst.wE j ∧ X.inst.wE j ≤ ((X.n : ℤ) + 1) ^ 650) ∧
      (∀ j, 0 < X.inst.wL j ∧ X.inst.wL j ≤ ((X.n : ℤ) + 1) ^ 650) ∧
      X.inst.d ≤ ((X.n : ℤ) + 1) ^ 650 ∧
      |X.Bnd| ≤ ((X.n : ℤ) + 1) ^ 650) :=
  ⟨X.reduction_iff, fun hn => RXC3.size_bound X hn⟩
```

Here `X : RXC3` is an arbitrary RXC3 instance with n elements and n sets, `X.inst` is the AWET instance of Section 8.1 with job type `Job X.n` ($8n$ state jobs and one anchor), `X.Bnd` is the decision bound $B = \lfloor A + \Gamma \rfloor$ of (8.34), and `X.inst.Yes B` means that some feasible schedule has cost at most B. The first conjunct is correctness (a); the second is the size bound (b), stated for $n \ge 1$ (for $n = 0$ the instance has a single job and $B = 0$).

1.3 The formal problem statement

A theorem is only as good as its definitions, so the definitions that appear in the main statement are kept short and close to Definition I.1.

```
/-- An AWET instance: positive integer processing times and weights. -/
structure Inst (J : Type) [Fintype J] where
  p : J → ℤ
  wE : J → ℤ
  wL : J → ℤ
  hp : ∀ j, 0 < p j
```

```
  hwE : ∀ j, 0 < wE j
  hwL : ∀ j, 0 < wL j

/-- The boundary nonrestrictive common due date `d = Σ p_j`. -/
def d (I : Inst J) : ℤ := ∑ j, I.p j

/-- A feasible nonpreemptive single-machine schedule, given by (real) completion times:
nonnegative start times and no two jobs overlapping. -/
structure Sched (I : Inst J) where
  C : J → ℝ
  start_nonneg : ∀ j, (I.p j : ℝ) ≤ C j
  no_overlap : ∀ i j, i ≠ j → C i + I.p j ≤ C j ∨ C j + I.p i ≤ C i

/-- Total weighted earliness-tardiness cost. -/
def cost (I : Inst J) (σ : I.Sched) : ℝ :=
  ∑ j, ((I.wE j : ℝ) * max ((I.d : ℝ) - σ.C j) 0 + (I.wL j : ℝ) * max (σ.C j - (I.d : ℝ)) 0)

/-- Decision version: is there a feasible schedule of cost at most `B`? -/
def Yes (I : Inst J) (B : ℤ) : Prop := ∃ σ : I.Sched, I.cost σ ≤ (B : ℝ)
```

- A schedule is given by real completion times C_j; the start time $C_j - p_j$ must be nonnegative, and the processing intervals of any two jobs must not overlap (one machine, no preemption). Idle time is allowed anywhere and no job order is fixed in advance, so `Yes` quantifies over all feasible schedules of Definition I.1, not only over canonical or integral ones.
- `cost` is exactly $F = \sum_j w_j^E (d - C_j)^+ + w_j^L (C_j - d)^+$ with $d = \sum_j p_j$.
- The job set is any finite type `J`; the linear order on it is only used to enumerate unordered pairs.

The source problem is Definition I.5:

```
/-- An RXC3 instance (Definition I.5): elements `0,…,n-1`, sets `S_0,…,S_{n-1}`; set `v` is
`{S v 0 < S v 1 < S v 2}` and every element lies in exactly three sets. -/
structure RXC3 where
  n : ℕ
  S : Fin n → Fin 3 → Fin n
  mono : ∀ v, S v 0 < S v 1 ∧ S v 1 < S v 2
  reg : ∀ k : Fin n, (univ.filter (fun q : Fin n × Fin 3 => S q.1 q.2 = k)).card = 3

/-- `x` selects an exact cover: every element lies in exactly one selected set. -/
def IsExactCover (x : Fin X.n → Bool) : Prop :=
  ∀ k : Fin X.n, (univ.filter (fun v => x v = true ∧ ∃ h, X.S v h = k)).card = 1

def HasExactCover : Prop := ∃ x, X.IsExactCover x

/-- prefix datum `d_{v,k} = |S_v ∩ {e_1,…,e_k}|` (elements are `0,…,n-1`, so the first `k`
elements are those with index `< k`). -/
def dpre (v : Fin X.n) (k : ℕ) : ℕ := (univ.filter (fun h : Fin 3 => (X.S v h : ℕ) < k)).card

/-- prefix count `C_k(x) = Σ_v d_{v,k} x_v` -/
def Cpre (x : Fin X.n → Bool) (k : ℕ) : ℤ := ∑ v, (X.dpre v k : ℤ) * (if x v then 1 else 0)

/-- the RPE equations `C_k(x) = k` (Definition I.6) -/
```

```
def RPE (x : Fin X.n → Bool) : Prop := ∀ k, 1 ≤ k → k ≤ X.n → X.Cpre x k = k
```

Elements and sets are numbered from 0; set v is $\{S_{v,0} < S_{v,1} < S_{v,2}\}$, and `reg` says that every element occurs in exactly three of the $3n$ incidences. The formalization does not forbid two sets from being equal, so the theorem covers slightly more inputs than Definition I.5 requires. `dpre`, `Cpre` and `RPE` are $d_{v,k}$, $C_k(x)$ and the prefix equations of Definition I.6 (the first k elements are those with index $< k$).

1.4 What the check rests on

- **Kernel and library.** The file is checked by Lean's kernel against Mathlib. `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound` for the main theorem, for the canonical reformulation and for the examples. The file declares no axioms and uses no `sorry`, `admit`, `native_decide`, `implemented_by` or `unsafe` code.
- **Independent replay.** `leanchecker` and `leanchecker-paranoid` (shipped with the toolchain) replay every declaration of the compiled module through the kernel, independently of the elaborator; both succeed.
- **Fidelity of the construction.** The definitions of Section 3 of the file transcribe Section 8.1 formula by formula (Table 1). As an external check, kernel-evaluated theorems in Section 16 of the file reproduce exact numbers printed in the paper's Appendix B (Tables 10 and 13), and the main theorem decides both worked examples correctly (Step 14).

2 Architecture of the formal proof

The formal proof follows the paper's chain of reasoning: canonical reformulation → construction → three gates (anchor early, every block coded, every prefix exact) → master identity and threshold. Figure 1 shows how the main formal results depend on each other. Where the paper uses a geometric or analytic argument that is awkward to formalize (schedule surgery in Lemma I.3, the profile integral in (8.10)), the formal proof uses an exact discrete replacement. These places are marked "Difference from the paper" in Section 3; none of them changes the mathematics of the reduction.

3 The proof, step by step

Each step below states what the paper claims, shows the Lean statement (proof bodies are elided and the comment gives their length; a few long lines are re-wrapped to fit the page), and explains how the formal proof goes. Lean names refer to `AWET_PartI.lean`; Appendix B lists where each part sits in the file.

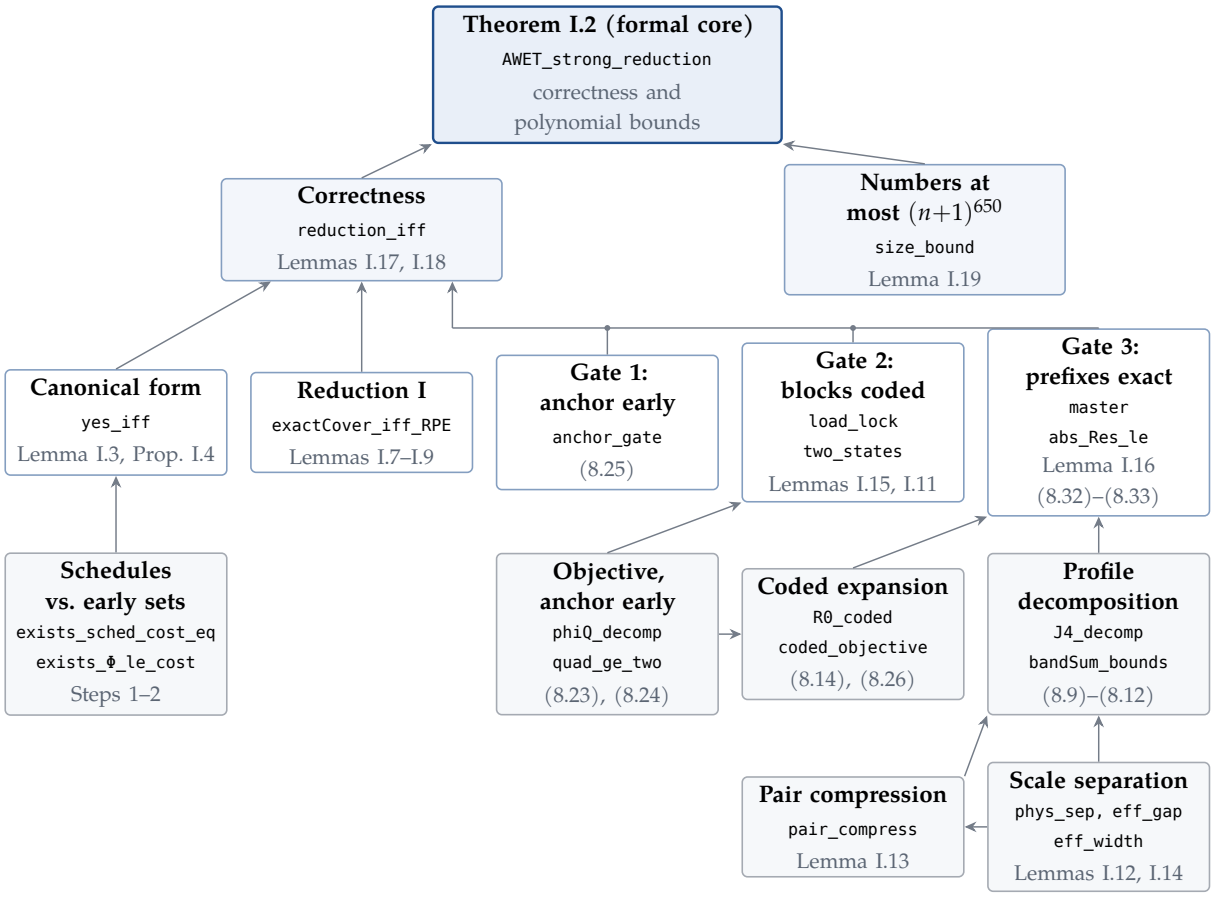


Figure 1: Dependency structure of the formal proof. Each box names the main Lean result and the corresponding item of the paper; an arrow means "is used to prove".

3.1 Step 1 — Canonical schedules realize $\Phi(E)$ (Proposition I.4)

Proposition I.4 For a nonempty early set E with $T = J \setminus E$, the due-date-aligned canonical schedule (early jobs before d in ratio order, tardy jobs after d in ratio order) is feasible and costs

$$\Phi(E) = \sum_{\{i,j\}\subseteq E} p_i p_j \min\{r_i, r_j\} + \sum_{\{i,j\}\subseteq T} p_i p_j \min\{s_i, s_j\} + \sum_{i\in T} p_i^2 s_i, \qquad r_i = \frac{w_i^E}{p_i},\ s_i = \frac{w_i^L}{p_i}.$$

In Lean the pair coefficients are written without division, since $p_i p_j \min\{r_i, r_j\} = \min\{p_j w_i^E,\ p_i w_j^E\}$:

```
/-- early pair coefficient `p_i p_j min(r_i, r_j)` written without division -/
def eP (I : Inst J) (i j : J) : ℤ := min (I.p j * I.wE i) (I.p i * I.wE j)

/-- tardy pair coefficient `p_i p_j min(s_i, s_j)` -/
def tP (I : Inst J) (i j : J) : ℤ := min (I.p j * I.wL i) (I.p i * I.wL j)

/-- The canonical set objective `Φ(E)` of Proposition I.4 (equation (SO)). -/
def Φ (I : Inst J) (E : Finset J) : ℤ :=
```

```
  (∑ i, ∑ j, if i < j then
      ((if i ∈ E ∧ j ∈ E then I.eP i j else 0) + (if i ∉ E ∧ j ∉ E then I.tP i j else 0))
    else 0)
  + ∑ i, if i ∈ E then 0 else I.p i * I.wL i
```

The canonical schedule is defined by explicit integer completion times. An early job j finishes at d minus the processing of the early jobs placed after it (those with a larger early ratio, ties broken by index); a tardy job finishes at $d + p_j$ plus the processing of the tardy jobs placed before it:

```
/-- Completion times of the canonical schedule of `E`: the early jobs end at `d` in
nondecreasing order of early ratio towards `d`, the tardy jobs start at `d` in nonincreasing
order of tardy ratio. -/
def canC (E : Finset J) (j : J) : ℤ :=
  if j ∈ E then I.d - ∑ i ∈ E.filter (fun i => I.keyE j < I.keyE i), I.p i
  else I.d + I.p j + ∑ i ∈ Eᶜ.filter (fun i => I.keyL j < I.keyL i), I.p i

/-- **Proposition I.4 (feasibility part).** Every early set `E` is realised by a feasible
schedule whose cost is exactly `Φ(E)`. -/
theorem exists_sched_cost_eq (E : Finset J) : ∃ σ : I.Sched, I.cost σ = (I.Φ E : ℝ) :=
  …  -- 1-line term proof
```

How the proof goes. Feasibility (`canC_start`, `canC_overlap`) is bookkeeping with these prefix sums. For the cost, each job contributes w_j^E times the processing of the early jobs closer to d (or w_j^L times its own and the earlier tardy processing); regrouping by pairs gives $\Phi(E)$, because when j is closer to d than i the pair contributes $p_j w_i^E = \min\{p_j w_i^E, p_i w_j^E\}$ (`eP_of_keyE_lt`). This is exactly the exchange-order fact used in the paper. A small generic lemma (`sum_key_lt_eq`) re-indexes a pair sum along any injective key.

Difference from the paper The formal statement also allows $E = \emptyset$ (all jobs tardy, the first one starting at d, which is feasible because $d \geq 0$). This is harmless and slightly simplifies the next step.

3.2 Step 2 — Every schedule costs at least $\min_E \Phi(E)$ (Lemma I.3, (6.1))

Lemma I.3 and (6.1) Some optimal schedule has no idle time, a completion at d, and both sides in ratio order; hence OPT $= \min_E \Phi(E)$. The paper proves the structural lemma by closing idle intervals, shifting a compact block to put a completion at d, and adjacent exchanges.

Formalizing these schedule-surgery arguments for real-valued schedules would be laborious. The formal proof instead establishes the inequality that the paper actually uses: every feasible schedule costs at least $\Phi(E)$ for some early set E.

```
/-- Packing: pairwise disjoint processing intervals inside `[a, b]` have total length at most
`b - a`. -/
theorem Sched.pack (σ : I.Sched) (a : ℝ) (F : Finset J) :
    ∀ b : ℝ, a ≤ b → (∀ i ∈ F, a + I.p i ≤ σ.C i ∧ σ.C i ≤ b) →
      (∑ i ∈ F, (I.p i : ℝ)) ≤ b - a := by
```

```
  …  -- 15 lines of tactic proof

/-- the completion-order cost expression of an early set -/
noncomputable def ordCost (σ : I.Sched) (E : Finset J) : ℝ :=
  ∑ j, if j ∈ E then (I.wE j : ℝ) * ∑ i ∈ E.filter (fun i => σ.C j < σ.C i), (I.p i : ℝ)
       else (I.wL j : ℝ) * ((I.p j : ℝ) + ∑ i ∈ Eᶜ.filter (fun i => σ.C i < σ.C j), (I.p i : ℝ))

theorem Φ_le_ordCost (σ : I.Sched) (E : Finset J) : (I.Φ E : ℝ) ≤ ordCost σ E := by
  …  -- 57 lines of tactic proof

/-- **Lemma I.3 / Proposition I.4 (optimality part).** Every feasible schedule costs at least
`Φ(E)` for some early set `E`. -/
theorem exists_Φ_le_cost (σ : I.Sched) : ∃ E : Finset J, (I.Φ E : ℝ) ≤ I.cost σ := by
  …  -- 171 lines of tactic proof

/-- **Canonical reformulation (Lemma I.3 + Proposition I.4).** The AWET decision problem with
bound `B` has a yes-answer iff some early set `E` has canonical cost `Φ(E) ≤ B`. -/
theorem yes_iff (B : ℤ) : I.Yes B ↔ ∃ E : Finset J, I.Φ E ≤ B := by
  …  -- 7 lines of tactic proof
```

How the proof goes. Fix a schedule σ and let $E_0 = \{j : C_j \le d\}$.

(i) *Packing* (`Sched.pack`): if the processing intervals of a set F of jobs lie inside $[a, b]$, then $\sum_{i \in F} p_i \le b - a$ (induction on F, removing the job that completes last).

(ii) `ordCost` is the cost of the pair structure induced by the completion order of σ. Every pair term of the form $p_j w_i^E$ is at least $\min\{p_j w_i^E,\ p_i w_j^E\}$, hence $\Phi(E) \le \mathrm{ordCost}(\sigma, E)$ for every E (`Φ_le_ordCost`).

(iii) If no job is tardy, packing the early jobs that complete after j into $[C_j, d]$ gives $w_j^E (d - C_j) \ge w_j^E \sum_{i \in E_0,\, C_i > C_j} p_i$, hence $\mathrm{cost}(\sigma) \ge \mathrm{ordCost}(\sigma, E_0) \ge \Phi(E_0)$.

(iv) Otherwise let j^* be the tardy job that completes first and let $\theta^+ = \max\{0,\ d - (C_{j^*} - p_{j^*})\} \in [0, p_{j^*}]$ be the part of j^* that straddles d. Packing gives, for early j and tardy j respectively,

$$d - C_j \ \ge \sum_{\substack{i \in E_0 \\ C_i > C_j}} p_i + \theta^+, \qquad C_j - d \ \ge \ p_j + \sum_{\substack{i \notin E_0 \\ C_i < C_j}} p_i - \theta^+,$$

so $\mathrm{cost}(\sigma) \ge \mathrm{ordCost}(\sigma, E_0) + \theta^+ (W_E - W_L)$ with $W_E = \sum_{E_0} w_j^E$ and $W_L = \sum_{j \notin E_0} w_j^L$.

(v) If $W_E \ge W_L$, drop the last term. If $W_E < W_L$, move j^* to the early side: an exact identity gives $\mathrm{ordCost}(\sigma, E_0 \cup \{j^*\}) = \mathrm{ordCost}(\sigma, E_0) + p_{j^*}(W_E - W_L)$, which is at most $\mathrm{ordCost}(\sigma, E_0) + \theta^+ (W_E - W_L)$ because $\theta^+ \le p_{j^*}$ and $W_E - W_L < 0$.

Together with Step 1 this gives `yes_iff`: *AWET with bound B is a yes-instance if and only if $\Phi(E) \le B$ for some early set E.* This is the only place where real-valued schedules appear; everything downstream works with finite sets of jobs and exact integer or rational arithmetic. It is also the formal content of the NP-membership argument after Theorem I.2: the early set E is a certificate.

Difference from the paper The formal proof does not use the structural Lemma I.3 itself (idle-time removal, alignment with d, adjacent exchanges). It replaces it by the lower bound above, which is affine in the straddle θ^+ and therefore needs no sorting of the schedule. The conclu-

sion used later, $\mathrm{OPT} = \min_E \Phi(E)$, is the same.

3.3 Step 3 — Reduction I: exact cover $\iff$ prefix exactness (Lemmas I.7–I.9)

Lemmas I.7–I.9 With $d_{v,k} = |S_v \cap \{e_1, \dots, e_k\}|$ and $C_k(x) = \sum_v d_{v,k} x_v$: $C_k(x) - C_{k-1}(x) = \sum_{v:\, e_k \in S_v} x_v$, and x selects an exact cover iff $C_k(x) = k$ for $k = 1, \dots, n$.

```
/-- **Lemma I.7 (prefix-difference identity).** -/
theorem Cpre_succ (x : Fin X.n → Bool) (k : ℕ) :
    X.Cpre x (k + 1) - X.Cpre x k
      = (univ.filter (fun v => x v = true ∧ ∃ h, (X.S v h : ℕ) = k)).card := by
  …  -- 6 lines of tactic proof

/-- regularity: the first `k` elements carry `3k` incidences -/
theorem sum_dpre (k : ℕ) (hk : k ≤ X.n) : ∑ v, (X.dpre v k : ℤ) = 3 * k := by
  …  -- 15 lines of tactic proof

/-- **Lemmas I.8 and I.9.** An exact cover is exactly a solution of the prefix equations. -/
theorem exactCover_iff_RPE (x : Fin X.n → Bool) : X.IsExactCover x ↔ X.RPE x := by
  …  -- 29 lines of tactic proof

theorem n_ge_three (hn : 0 < X.n) : 3 ≤ X.n := by
  …  -- 5 lines of tactic proof
```

How the proof goes. `Cpre_succ` splits the filter defining $d_{v,k+1}$ into $d_{v,k}$ plus the indicator of "element k lies in S_v". Both directions of `exactCover_iff_RPE` then telescope. `sum_dpre` (regularity: the first k elements carry $3k$ incidences) is used later in (8.13). `n_ge_three` records that a nonempty instance automatically has $n \geq 3$, which is the standing assumption of Section 8. The formal reduction goes directly from RXC3 to AWET, so Corollary I.10 (NP-completeness of RPE) is not needed.

3.4 Step 4 — The construction of Section 8.1

The construction is transcribed definition by definition; Table 1 maps the paper's symbols to Lean names. Indices are 0-based: block $v \in \{0, \dots, n-1\}$ is the paper's block $v+1$, and pair $h \in \{0,1,2,3\}$ of a block is the controller ($h = 0$) or the occurrence pair of element $S_{v,h-1}$. A state job is a triple (v, h, b): $b =$ `false` is the heavy job A_{vh} and $b =$ `true` the light job B_{vh} of Table 2.

Two excerpts show the style; the full list is in Section 3 of the file.

```
/-- `N = n + 1` -/
def NN : ℤ := X.n + 1

def G : ℤ := X.NN ^ 20

def M : ℤ := X.NN ^ 40

def T : ℤ := X.NN ^ 50
```

Paper	Lean	Definition in the file
$N,\ G,\ M,\ T$	`NN, G, M, T`	$n+1,\ N^{20},\ N^{40},\ N^{50}$
$Q_0,\dots,Q_3$	`Qc h`	10^{h+1}
$\kappa(k_{vh})$	`ht v h`	0 for the controller, $n - S_{v,h-1}$ for an occurrence pair ("height")
P_{vh} (8.1)–(8.2)	`P v h`	$T + M \cdot \mathrm{ht} + Q_h$
p_i, ϵ_i (Table 2)	`pS, eps`	$P_{vh}+3$ / P_{v0} (controller), $P_{vh}+1$ / P_{vh} (occurrence); signs of Table 2
β_v	`β v`	$\sum_h P_{vh} + 3$
$\lambda_v,\ L_v,\ K_{uv}$	`lam, L, K`	$G+n-v$ $(= G+n-v'+1$ for $v'=v+1)$, $2\lambda_v$, $\min\{L_u, L_v\}$
$f = K\beta$	`f v`	$\sum_u K_{vu}\beta_u$
r_i (8.5), w_i^E	`rS, wES`	$f_v + \lambda_v p_i,\ p_i r_i$
ρ_{vh}	`ρ v h`	$f_v + \lambda_v(2P_{vh}+3)$ / $f_v + \lambda_v(2P_{vh}+1)$
$R_k = [\underline{\rho}_k, \overline{\rho}_k]$	`lo η, hi η`	min / max of the effective ratios at height η
Δ_k (8.6)–(8.7), W	`Δ k, W`	`lo (n-k+1) - hi (n-k)`; $\sum_{\eta=0}^{n}(\mathrm{hi}_\eta - \mathrm{lo}_\eta)$
α_{ij}, a_v^0 (8.14)	`α, a0`	$\min\{p_j w_i^E, p_i w_j^E\}$; the baseline field (within-block pairs as half the ordered sum)
$a_v^\star$ (8.15), D_v (8.16)	`astar, Dv`	$\frac{1}{4}\sum_k \Delta_k k d_{v,k}$; $\sum_{i\in B_v} \epsilon_i p_i^2$
η_v (8.17), e_v (8.18)	`eta, ev`	$\mathrm{round}(4(a_v^0 - a_v^\star)/D_v)$ with round $x = \lfloor x + \frac{1}{2} \rfloor$; $a_v^0 - \eta_v D_v/4 - a_v^\star$
$\bar{\eta}_{ij}, U$ (8.21), H (8.22)	`etabar, U, H`	as in the paper; $H = U + 2\max_v \lvert\eta_v\rvert + 2$
s_i (8.20), w_i^L	`sS, wLS`	$HL_v + \eta_v,\ p_i s_i$
anchor a	`Job.anchor`	$p_a = H,\ w_a^E = H(1+\max_i r_i),\ w_a^L = Hs_a$
$C^\star, s_a$ (8.25)	`Cstar, sa`	Φ of the coded set $\sigma \equiv +1$ with the anchor early; $1 + \lceil C^\star/H^2 \rceil$
J_{uv} (8.10), C_0 (8.26)	`Jc, C0`	$\frac{1}{4}\sum \epsilon_i\epsilon_j\alpha_{ij}$; $C^\star - \sum_v(a_v^\star + e_v) - \sum_{u<v} J_{uv}$
A (8.29), Γ (8.32)	`Aconst, Gam`	$C_0 - \frac{1}{8}\sum_k \Delta_k(k^2 + \sum_v d_{v,k}^2)$; $\frac{9}{8}n^2W + \frac{1}{8}\sum_v D_v$
B (8.34)	`Bnd`	$\lfloor A + \Gamma \rfloor$
AWET instance	`inst`	$8n$ state jobs `Job.st (v,h,b)` and the anchor

Table 1: The construction of Section 8.1 and its Lean counterparts.

```
/-- pair codes `Q_0,…,Q_3 = 10, 100, 1000, 10000` -/
def Qc (h : Fin 4) : ℤ := 10 ^ (h.val + 1)

/-- height of pair `h` of block `v`: `0` for the controller, the reversed level `κ(k) = n - k`
(0-based element `k`) for the occurrence pair of element `S v (h-1)`. -/
def ht (v : Fin X.n) (h : Fin 4) : ℕ :=
  if hh : h.val = 0 then 0 else X.n - (X.S v ⟨h.val - 1, by omega⟩ : ℕ)

/-- base processing values (8.1)-(8.2): `P_{vh} = T + M κ + Q_h` (`κ = 0` for the controller) -/
def P (v : Fin X.n) (h : Fin 4) : ℤ := X.T + X.M * X.ht v h + Qc h

/-- heavy increment: 3 for the controller pair, 1 for an occurrence pair -/
def cA (h : Fin 4) : ℤ := if h.val = 0 then 3 else 1

/-- reference sign `ε` (Table 2); `b = false` is the heavy job `A`, `b = true` the light job
`B` -/
def eps (h : Fin 4) (b : Bool) : ℤ := (if h.val = 0 then -1 else 1) * (if b then -1 else 1)
```

```
/-- processing time of a state job (Table 2) -/
def pS (i : SJ X.n) : ℤ := X.P i.1 i.2.1 + (if i.2.2 then 0 else cA i.2.1)

/-- target load `β_v` -/
def β (v : Fin X.n) : ℤ := (∑ h, X.P v h) + 3
```

```
def Job (n : ℕ) : Type := Option (SJ n)

/-- job data shared by the preliminary and final instances -/
def pJ (j : Job X.n) : ℤ := j.elim X.H X.pS

def wEJ (j : Job X.n) : ℤ := j.elim (X.H * (1 + X.rmax)) X.wES

/-- the coded early set of a spin vector: the anchor and every state job with `σ_v ε_i = +1` -/
def codedE (σ : Fin X.n → Bool) : Finset (Job X.n) :=
  univ.filter (fun j => j.elim true (fun i => decide (sgn (σ i.1) * eps i.2.1 i.2.2 = 1)) =
      true)

/-- the benchmark cost `C⋆` of the coded assignment `σ ≡ +1` with the anchor early -/
def Cstar : ℤ := X.preInst.Φ (X.codedE (fun _ => true))

/-- anchor tardiness ratio (8.25) -/
def sa : ℤ := 1 + ⌈(X.Cstar : ℚ) / (X.H : ℚ) ^ 2⌉

/-- **The constructed AWET instance** (Section 8.1): `8n` state jobs and one anchor. -/
def inst : Inst (Job X.n) where
  p := X.pJ
  wE := X.wEJ
  wL := fun j => j.elim (X.H * X.sa) X.wLS
  hp := X.pJ_pos
  hwE := X.wEJ_pos
  hwL := fun j => by
    rcases Job.cases j with rfl | ⟨i, rfl⟩
    · simp only [Job.elim_anchor]; exact mul_pos X.H_pos X.sa_pos
    · simp [X.wLS_pos i]
```

Formalization details (i) $C^\star$ is defined through a *preliminary* instance `preInst` whose anchor tardiness weight is a placeholder, because s_a (hence w_a^L) depends on $C^\star$. `Φ_pre_eq_inst` proves that both instances give the same $\Phi(E)$ whenever the anchor is early, which is the paper's remark that "this cost does not depend on the anchor's tardiness ratio". (ii) Where the paper leaves a choice open, the formalization fixes one: the benchmark coded assignment for $C^\star$ is $\sigma \equiv +1$, ties in (8.17) round up, and C_0 is read off from the benchmark assignment (the theorem `coded_objective` below shows that this is the constant coefficient of (8.26)).

3.5 Step 5 — Two zero-load states (Lemma I.11 and (8.4))

Lemma I.11, (8.4) For a sign vector c on block v, $\sum_{i\in B_v} p_i c_i = 0$ iff $c = \pm\epsilon$. Equivalently, the tardy load of block v is $b_v = \beta_v$ iff the block is in one of its two coded orientations.

```
/-- the digit-decoding step of Lemma I.11 -/
theorem digit_decode (T M U Z Qs res B : ℤ) (hM0 : 0 ≤ M) (hT : B * M + 11113 < T)
    (hM : 11113 < M) (hU : U ≠ 0 → 1 ≤ |U|) (hZ : |Z| ≤ B) (hQ : |Qs| ≤ 11110) (hres : |res| ≤
        3)
    (heq : T * U + M * Z + Qs + res = 0) : U = 0 ∧ Z = 0 ∧ Qs + res = 0 := by
  …  -- 22 lines of tactic proof

/-- **Lemma I.11.** A block whose tardy processing equals `β_v` is in one of the two coded
orientations. -/
theorem two_states (hn : 0 < X.n) (v : Fin X.n) (early : Fin 4 × Bool → Bool)
    (hb : ∑ a, (if early a then 0 else X.pS (v, a)) = X.β v) :
    ∃ s : Bool, ∀ a, early a = decide (sgn s * eps a.1 a.2 = 1) := by
  …  -- 119 lines of tactic proof

theorem bq_coded (σ : Fin X.n → Bool) (u : Fin X.n) : X.bq (X.xE (X.codedE σ)) u = X.β u := by
  …  -- 11 lines of tactic proof
```

How the proof goes. `two_states` takes an arbitrary early/tardy pattern `early` of the eight jobs of block v with tardy load β_v and returns the spin s of the block. Writing the load equation in the digits of T, M and the pair codes, the abstract lemma `digit_decode` peels off the T digit and then the M digit (both use only $|\text{lower terms}| < \text{digit}$, from $3nM + 11113 < T$ and $11113 < M$). What is left is the code equation for the four pairs; `pair_decode` solves it by exhaustive case analysis over the 2^8 early/tardy patterns (each case closed by `omega`), showing that only the two coded orientations survive. The converse (`bq_coded`) is a direct computation.

3.6 Step 6 — Kernel field and scale separation (Lemmas I.12 and I.14)

Lemmas I.12, I.14 $4T < \beta_v < 5T$; f decreases and $0 < f_1 - f_n < 10N^2T$. The physical ratio intervals of complete clusters are disjoint and ordered like their effective atoms; the bands satisfy $R_1 > \cdots > R_n > R_0$, every main gap Δ_k exceeds GM, and $W < 15N^3T$.

```
theorem β_bounds (v : Fin X.n) : 4 * X.T < X.β v ∧ X.β v < 5 * X.T := by
  …  -- 12 lines of tactic proof

/-- **Lemma I.12** (consequences used later): the kernel field is decreasing in the block index,
strictly, and its total variation is `< 10 n² T`. -/
theorem f_sub_bounds {u w : Fin X.n} (huw : u ≤ w) :
    0 ≤ X.f u - X.f w ∧ X.f u - X.f w ≤ 10 * (X.n : ℤ) ^ 2 * X.T := by
  …  -- 27 lines of tactic proof

/-- the master numerical inequality behind all separations:
`10 n² T + 2 n T + 2 n² M + 20003 (G + n) < G M` -/
```

```
theorem scale_key :
    10 * (X.n : ℤ) ^ 2 * X.T + 2 * X.n * X.T + 2 * (X.n : ℤ) ^ 2 * X.M + 20003 * (X.G + X.n)
      < X.G * X.M := by
  …  -- 32 lines of tactic proof

def valE (t : ℤ) (u : Fin X.n) (a Q c : ℤ) : ℤ := X.f u + X.lam u * (t * (X.T + X.M * a + Q) +
    c)

def above (u : Fin X.n) (g : Fin 4) (v : Fin X.n) (h : Fin 4) : Prop :=
  X.ht v h < X.ht u g ∨ (X.ht u g = X.ht v h ∧ u < v)

/-- physical separation of clusters (first half of Lemma I.14) -/
theorem phys_sep {u v : Fin X.n} {g h : Fin 4} (hab : X.above u g v h) (b b' : Bool) :
    X.rS (v, h, b') < X.rS (u, g, b) := by
  …  -- 12 lines of tactic proof

/-- effective separation of clusters -/
theorem eff_sep {u v : Fin X.n} {g h : Fin 4} (hab : X.above u g v h) : X.ρ v h < X.ρ u g := by
  …  -- 10 lines of tactic proof

/-- the main gap: consecutive heights are separated by more than `G M` -/
theorem eff_gap {u v : Fin X.n} {g h : Fin 4} (hht : X.ht v h + 1 = X.ht u g) :
    X.ρ v h + X.G * X.M < X.ρ u g := by
  …  -- 5 lines of tactic proof

/-- band width bound -/
theorem eff_width {u v : Fin X.n} {g h : Fin 4} (hht : X.ht u g = X.ht v h) :
    |X.ρ u g - X.ρ v h| ≤ X.Wb := by
  …  -- 3 lines of tactic proof
```

How the proof goes. All physical early ratios ($t = 1$) and effective ratios ($t = 2$) have the common form `valE t u a Q c` $= f_u + \lambda_u\big(t(T + Ma + Q) + c\big)$, so the separation arguments are proved once for both. The relation `above u g v h` says that cluster (u, g) is at a larger height than (v, h), or at the same height with a smaller block index. `phys_sep` and `eff_sep` show that both the physical intervals and the effective atoms respect this order, which is the agreement of orders required by Lemma I.13. `eff_gap` and `eff_width` give the gap and width bounds. All numerical comparisons reduce to the single inequality `scale_key`, which holds because $N \geq 4$. The band-level consequences are `gap_gt` ($\Delta_k > GM$) and `W_le` ($W \leq 15N^{53} = 15N^3T$).

3.7 Step 7 — Exact pair compression and the profile decomposition (Lemma I.13, (8.9)–(8.12))

Lemma I.13, (8.10)–(8.12) For complete clusters g, h of different blocks whose orders agree, $I^{\mathrm{phys}}(g, h) = m_g m_h \min\{\rho_g, \rho_h\}$. Hence $J_{uv} = \frac{1}{4}\sum_{g,h} m_{ug} m_{vh} \min\{\rho_{ug}, \rho_{vh}\} = \frac{1}{4}\int_0^\infty F_u(t) F_v(t)\, dt$, and on the k-th main gap the profile equals $F_v = d_{v,k}$.

```
/-- **Lemma I.13 (exact pair compression).** For clusters of different blocks the four signed
physical early interactions equal `m_g m_h min(ρ_g, ρ_h)`. -/
theorem pair_compress (hn : 0 < X.n) {u v : Fin X.n} (huv : u ≠ v) (g h : Fin 4) :
    ∑ b : Bool, ∑ b' : Bool, eps g b * eps h b' * X.α (u, g, b) (v, h, b')
      = mass g * mass h * min (X.ρ u g) (X.ρ v h) := by
  …  -- 22 lines of tactic proof

def J4 (u v : Fin X.n) : ℤ :=
  ∑ a : Fin 4 × Bool, ∑ b : Fin 4 × Bool, eps a.1 a.2 * eps b.1 b.2 * X.α (u, a) (v, b)

theorem J4_compress (hn : 0 < X.n) {u v : Fin X.n} (huv : u ≠ v) :
    X.J4 u v = ∑ g : Fin 4, ∑ h : Fin 4, mass g * mass h * min (X.ρ u g) (X.ρ v h) := by
  …  -- 7 lines of tactic proof
```

The integral in (8.10) is replaced by an exact finite identity. Every atom position $x \in [\mathrm{lo}_0, \mathrm{hi}_n]$ is split into its portions inside the bands and inside the gaps,

$$x = \mathrm{lo}_0 + \sum_{\eta=0}^{n} \mathrm{bp}_\eta(x) + \sum_{\eta=0}^{n-1} \mathrm{gp}_\eta(x),$$

where bp_η and gp_η clamp x to band η and to gap η. Clamping commutes with min, so $\min\{\rho, \rho'\}$ splits layer by layer:

```
def clampZ (x a b : ℤ) : ℤ := max a (min x b)

/-- band part and gap part of a point -/
def bp (η : ℕ) (x : ℤ) : ℤ := clampZ x (X.lo η) (X.hi η) - X.lo η

def gp (η : ℕ) (x : ℤ) : ℤ := clampZ x (X.hi η) (X.lo (η + 1)) - X.hi η

theorem layer_decomp {x : ℤ} (h1 : X.lo 0 ≤ x) (h2 : x ≤ X.hi X.n) :
    x = X.lo 0 + ∑ η ∈ range (X.n + 1), X.bp η x + ∑ η ∈ range X.n, X.gp η x := by
  …  -- 3 lines of tactic proof

theorem Mab_eq_dpre {η : ℕ} (hη : η < X.n) (u : Fin X.n) : X.Mab u η = X.dpre u (X.n - η) := by
  …  -- 13 lines of tactic proof

/-- each band contributes between `0` and `9 × width` -/
theorem bandSum_bounds {η : ℕ} (hη : η ≤ X.n) (u v : Fin X.n) :
    0 ≤ X.bandSum η u v ∧ X.bandSum η u v ≤ 9 * (X.hi η - X.lo η) := by
  …  -- 35 lines of tactic proof

/-- **Profile decomposition (8.10)-(8.12)**: `4 J_{uv}` = main-gap part + band part. -/
theorem J4_decomp {u v : Fin X.n} (huv : u ≠ v) :
    X.J4 u v = ∑ η ∈ range X.n, X.gap η * X.dpre u (X.n - η) * X.dpre v (X.n - η)
      + ∑ η ∈ range (X.n + 1), X.bandSum η u v := by
  …  -- 33 lines of tactic proof
```

How the proof goes. On gap η an atom contributes the full gap length if it lies above the gap and 0 otherwise (`gp_atom`); the signed mass of block u above gap η is $d_{u,n-\eta}$ (`Mab_eq_dpre`), which is (8.11). Summing over atom pairs gives the main-gap term $\sum_\eta \mathrm{gap}_\eta\, d_{u,n-\eta} d_{v,n-\eta}$, that is, $4J_{uv}^{\mathrm{main}}$. For the band term the formal proof uses one more fact: inside one band a block has at most one atom,

because the heights of a block are distinct. Hence the profile of a block on a band takes only the values M^{ab} and $M^{\mathrm{ab}} + M^{\mathrm{at}}$, which both lie in $[0, 3]$, and every band contributes between 0 and 9 times its width (`bandSum_bounds`).

Difference from the paper The paper bounds the band contribution through $|F_v(t)| \le 3$, which gives $|J_{uv} - J_{uv}^{\mathrm{main}}| \le \frac{9}{4}W$. The discrete argument gives the sharper $0 \le J_{uv} - J_{uv}^{\mathrm{main}} \le \frac{9}{4}W$ (`Jc_sub_Jmain`). Only the absolute bound is needed.

3.8 Step 8 — The objective with an early anchor and the load lock (Lemma I.15, (8.23)–(8.24))

Lemma I.15 With the anchor early, the dominant assignment-dependent part of the objective is $\frac{H}{2}(b-\beta)^\top K(b-\beta)$ (8.23), where b_v is the tardy load of block v; by (8.24) it is at least H for every integer $b \neq \beta$, while all other assignment-dependent terms oscillate by at most $U - 1 < H$. Hence every optimum with the anchor early has $b = \beta$.

First, the objective of any early set that contains the anchor is written in terms of state jobs only. The anchor's early ratio exceeds every state ratio, so each early state job contributes $p_a p_i r_i = H w_i^E$:

```
/-- **Formula (*)**: the canonical objective of an early set containing the anchor. -/
theorem Φ_anchor_early {I : Inst (Job X.n)} (hI : X.Agrees I) {E : Finset (Job X.n)}
    (hE : Job.anchor ∈ E) :
    I.Φ E = ∑ s : SJ X.n, (if Job.st s ∈ E then X.H * X.wES s else 0)
      + ∑ s : SJ X.n, ∑ t : SJ X.n, (if encS s < encS t then
          ((if Job.st s ∈ E ∧ Job.st t ∈ E then X.α s t else 0)
            + (if Job.st s ∉ E ∧ Job.st t ∉ E then X.τ s t else 0)) else 0)
      + ∑ s : SJ X.n, (if Job.st s ∈ E then 0 else X.pS s * X.wLS s) := by
  …  -- 22 lines of tactic proof
```

The same expression, read as a polynomial in the early indicators $x_s \in \{0,1\}$, is `phiQ`, and its exact decomposition is the formal version of (8.23):

```
/-- the canonical objective of an anchor-early set, as a polynomial in its early indicators -/
def phiQ (x : SJ X.n → ℚ) : ℚ :=
  ∑ s, x s * ((X.H : ℚ) * X.wES s)
    + X.psum (fun s t => x s * x t * X.α s t + (1 - x s) * (1 - x t) * X.τ s t)
    + ∑ s, (1 - x s) * ((X.pS s : ℚ) * X.wLS s)

/-- the residual `R₀` of Lemma I.15 -/
def R0 (x : SJ X.n → ℚ) : ℚ :=
  X.psum (fun s t => x s * x t * X.α s t)
    + X.psum (fun s t => (1 - x s) * (1 - x t) * ((X.pS s : ℚ) * X.pS t * X.etabar s t))
    + ∑ s, (1 - x s) * ((X.pS s : ℚ) ^ 2 * X.eta s.1)

/-- the load-lock quadratic `(b - β)ᵀ K (b - β)` -/
def lockQ (x : SJ X.n → ℚ) : ℚ :=
  ∑ u, ∑ v, (X.K u v : ℚ) * (X.bq x u - X.β u) * (X.bq x v - X.β v)
```

```
/-- **Lemma I.15, exact form**: on anchor-early sets, the objective is a constant plus the
dominant load-lock penalty `(H/2)(b-β)ᵀK(b-β)` plus the residual `R₀`. -/
theorem phiQ_decomp (hn : 0 < X.n) (x : SJ X.n → ℚ) (hx : ∀ s, x s = 0 ∨ x s = 1) :
    X.phiQ x = X.K0 + ((X.H : ℚ) / 2) * X.lockQ x + X.R0 x := by
  …  -- 93 lines of tactic proof
```

How the proof goes. The tardy pair coefficient splits exactly as $\tau_{st} = H\, p_s p_t K_{v(s)v(t)} + p_s p_t \bar{\eta}_{st}$ (`τ_eq`); this uses the tardy order $s_i > s_j$ for $\mathrm{blk}(i) < \mathrm{blk}(j)$ (`sS_strict`, `min_sS`). The H-terms of the tardy pairs, the tardy self terms and the anchor interactions combine into $\frac{H}{2}(b-\beta)^\top K(b-\beta)$ plus a constant; as in the paper, the square-processing terms q_v cancel exactly because $L_v = 2\lambda_v$ and $f = K\beta$. Everything else is collected in R_0. The positive definiteness (8.24) and the oscillation bound are:

```
/-- prefix-square representation (8.24) of the minimum-kernel quadratic form -/
theorem quad_prefix (y : Fin X.n → ℚ) :
    ∑ u, ∑ v, (X.K u v : ℚ) * y u * y v
      = ∑ k ∈ range X.n, ((X.Lnat k - X.Lnat (k + 1) : ℤ) : ℚ)
          * (∑ u : Fin X.n, if u.val ≤ k then y u else 0) ^ 2 := by
  …  -- 18 lines of tactic proof

/-- **Positive definiteness**: every nonzero integer load error costs at least `2`. -/
theorem quad_ge_two (y : Fin X.n → ℚ) (hint : ∀ u, ∃ z : ℤ, y u = z) (hne : ∃ u, y u ≠ 0) :
    2 ≤ ∑ u, ∑ v, (X.K u v : ℚ) * y u * y v := by
  …  -- 38 lines of tactic proof

theorem R0_bounds (x : SJ X.n → ℚ) (hx : ∀ s, x s = 0 ∨ x s = 1) :
    X.Rlo ≤ X.R0 x ∧ X.R0 x ≤ X.Rlo + ((X.U : ℚ) - 1) := by
  …  -- 33 lines of tactic proof
```

Here `Lnat` extends L by $L_{n+1} = 0$, and `quad_prefix` is (8.24) verbatim. For a nonzero integer vector y, the prefix sum ending at the first nonzero entry equals that entry, so the corresponding square is at least 1 and its coefficient at least 2 (`quad_ge_two`). `R0_bounds` is the statement $\mathrm{osc}(R_0) \le U - 1$: each term of R_0 is a coefficient times a $0/1$ indicator, and $U - 1$ is exactly the sum of the absolute values of these coefficients (`U_sub_one`).

3.9 Step 9 — Gates 1 and 2: the anchor is early, every block is coded ((8.25), Lemma I.15)

```
/-- **Lemma I.15 (global load lock).** With the anchor early, any early set whose tardy block
loads differ from `β` costs strictly more than the coded benchmark set. -/
theorem load_lock (hn : 0 < X.n) {E : Finset (Job X.n)} (hE : Job.anchor ∈ E)
    (hb : ∃ u, X.bq (X.xE E) u ≠ X.β u) :
    X.inst.Φ (X.codedE (fun _ => true)) < X.inst.Φ E := by
  …  -- 25 lines of tactic proof

/-- **Anchor gate (8.25).** An early set with the anchor tardy costs more than `C⋆`. -/
theorem anchor_gate {E : Finset (Job X.n)} (hE : Job.anchor ∉ E) :
    X.Cstar < X.inst.Φ E := by
  …  -- 33 lines of tactic proof
```

How the proof goes. `load_lock` compares an anchor-early set E with a wrong load against the benchmark coded set (spin $\sigma \equiv +1$, load exactly β, lock penalty 0). By `phiQ_decomp`, the difference of the two objectives is at least $\frac{H}{2} \cdot 2 - (U-1) = H - U + 1 > 0$. `anchor_gate`: if the anchor is tardy, its own tardy term alone is $p_a w_a^L = H^2 s_a = H^2\big(1 + \lceil C^\star/H^2 \rceil\big) > C^\star$, and all other terms of Φ are nonnegative. Combining `load_lock` with Lemma I.11 gives `coded_of_balanced`: a minimizer has the form $E = \mathrm{codedE}(\sigma)$ for a spin vector σ.

> **Difference from the paper** Lemma I.15 is stated in the paper for optimal canonical assignments. The formal `load_lock` is a pointwise strict comparison with one explicit coded set, which is slightly stronger and is exactly what the gate argument needs.

3.10 Step 10 — Coded expansion and the master identity (Lemma I.16, (8.26)–(8.31))

> **Lemma I.16** On coded states, $\Phi_{\mathrm{can}}(\sigma) = C_0 + \sum_v (a_v^\star + e_v)\sigma_v + \sum_{u<v} J_{uv}\sigma_u\sigma_v$ (8.26), and with $x_v = (1+\sigma_v)/2$,
>
> $$\Phi_{\mathrm{can}}(\sigma) = A + \mathcal{P}(x) + R(\sigma), \qquad \mathcal{P}(x) = \sum_{k=1}^{n} \frac{\Delta_k}{2}\big(C_k(x) - k\big)^2,$$
>
> $$R(\sigma) = \sum_v e_v\sigma_v + \sum_{u<v}\big(J_{uv} - J_{uv}^{\mathrm{main}}\big)\sigma_u\sigma_v, \qquad J_{uv}^{\mathrm{main}} = \frac{1}{4}\sum_{k=1}^{n} \Delta_k d_{u,k} d_{v,k}.$$

```
/-- **Coded expansion (8.26).** On coded assignments the residual of the load lock is a
constant plus the target field plus the field error plus the cross-block couplings. -/
theorem R0_coded (_hn : 0 < X.n) (σ : Fin X.n → Bool) :
    X.R0 (X.xE (X.codedE σ)) = X.Cexp + X.linC σ + X.quadC σ := by
  …  -- 129 lines of tactic proof

theorem coded_objective (hn : 0 < X.n) (σ : Fin X.n → Bool) :
    (X.inst.Φ (X.codedE σ) : ℚ) = X.C0 + X.linC σ + X.quadC σ := by
  …  -- 18 lines of tactic proof

/-- main-gap coupling (8.27) -/
def Jmain (u v : Fin X.n) : ℚ := (1/4) * ∑ k ∈ Icc 1 X.n, (X.Δ k : ℚ) * X.dpre u k * X.dpre v k

/-- prefix penalty `𝒫(x)` (8.28) -/
def Pen (x : Fin X.n → Bool) : ℚ := ∑ k ∈ Icc 1 X.n, ((X.Δ k : ℚ) / 2) * (X.Cpre x k - k) ^ 2

/-- residual `R(σ)` (8.30) -/
def Res (σ : Fin X.n → Bool) : ℚ := ∑ v, X.ev v * sgnQ (σ v)
  + ∑ u, ∑ v, if u < v then (X.Jc u v - X.Jmain u v) * sgnQ (σ u) * sgnQ (σ v) else 0

/-- **Lemma I.16 (exact master decomposition).** -/
theorem master (hn : 0 < X.n) (σ : Fin X.n → Bool) :
    (X.inst.Φ (X.codedE σ) : ℚ) = X.Aconst + X.Pen σ + X.Res σ := by
  …  -- 89 lines of tactic proof
```

How the proof goes. On a coded set the early indicator of state job s is $x_s = (1 + \sigma_{v(s)}\epsilon_s)/2$ (`xE_`

coded). Substituting this into R_0 and expanding gives a constant, a linear term and a quadratic term in σ (`R0_coded`). The coefficient of σ_v is the baseline field a_v^0 of (8.14) (written in ordered-pair form by `a0_eq`) minus the tardiness-offset contribution $\eta_v D_v/4$, which is $a_v^\star + e_v$ by the definition (8.18) of e_v. The coefficient of $\sigma_u\sigma_v$ is J_{uv}. Because the load-lock term vanishes on coded sets, this is (8.26) (`coded_objective`); evaluating it at $\sigma \equiv +1$ identifies the constant with C_0. `master` then completes the square prefix by prefix: $\frac{\Delta_k}{8}Y_k^2 + \frac{\Delta_k k}{4}Y_k = \frac{\Delta_k}{8}(Y_k + k)^2 - \frac{\Delta_k}{8}k^2$ with $Y_k = \sum_v d_{v,k}\sigma_v$, and $Y_k + k = 2(C_k(x) - k)$ by (8.13), which uses $\sum_v d_{v,k} = 3k$ (`sum_dpre`).

3.11 Step 11 — Residual budget and decision gap ((8.19), (8.32)–(8.34))

> **(8.19), (8.32), (8.33)** $|e_v| \le D_v/8$; $|R(\sigma)| \le \Gamma = \frac{9}{8}n^2W + \frac{1}{8}\sum_v D_v$; and $\Delta_{\min}/2 > 2\Gamma + 1$.

```
/-- `|e_v| ≤ D_v / 8` (8.19) -/
theorem abs_ev_le (v : Fin X.n) : |X.ev v| ≤ (X.Dv v : ℚ) / 8 := by
  …  -- 10 lines of tactic proof

theorem Jc_sub_Jmain (hn : 0 < X.n) {u v : Fin X.n} (huv : u ≠ v) :
    0 ≤ X.Jc u v - X.Jmain u v ∧ X.Jc u v - X.Jmain u v ≤ (9/4) * X.W := by
  …  -- 20 lines of tactic proof

/-- **Residual bound (8.32)**: `|R(σ)| ≤ Γ`. -/
theorem abs_Res_le (hn : 0 < X.n) (σ : Fin X.n → Bool) : |X.Res σ| ≤ X.Gam := by
  …  -- 33 lines of tactic proof

/-- `4 Γ < G M`, hence `Δ_min / 2 > 2 Γ` -/
theorem four_Gam_lt : 4 * X.Gam < (X.G : ℚ) * X.M := by
  …  -- 21 lines of tactic proof

theorem Δ_gt {k : ℕ} (hk : k ∈ Icc 1 X.n) : X.G * X.M < X.Δ k := by
  …  -- 3 lines of tactic proof

theorem Pen_ge {x : Fin X.n → Bool} {k : ℕ} (hk : k ∈ Icc 1 X.n) (hne : X.Cpre x k ≠ k) :
    (X.Δ k : ℚ) / 2 ≤ X.Pen x := by
  …  -- 15 lines of tactic proof
```

How the proof goes. `abs_ev_le` uses only $|\operatorname{round}(x) - x| \le \frac{1}{2}$ and $D_v > 0$. `abs_Res_le` adds the field errors and the band terms of Step 7 over at most $n^2/2$ block pairs (`count_lt`). For the gap, `four_Gam_lt` proves $4\Gamma < GM$ from the integer inequality $36n^2W + 4\sum_v D_v < 8GM$, using $W \le 15N^{53}$ and $D_v = 2M(\kappa_1 + \kappa_2 + \kappa_3) + 22134$ (`Dv_eq'`). With `Δ_gt` this gives $\Delta_{\min}/2 > 2\Gamma$. Finally `Pen_ge`: one violated prefix equation costs at least $\Delta_k/2$, because $C_k(x) - k$ is a nonzero integer.

3.12 Step 12 — Both directions (Lemmas I.17 and I.18)

This is the complete formal proof of the reduction for $n \ge 1$; each step of the paper's argument is visible as a few lines.

```
/-- **Theorem I.2 (correctness of the reduction).** For every RXC3 instance with `n ≥ 1`
(hence `n ≥ 3`), the instance has an exact cover iff the constructed AWET instance has a
schedule of cost at most `B`. -/
theorem reduction_correct (hn : 0 < X.n) : X.HasExactCover ↔ X.inst.Yes X.Bnd := by
  rw [Inst.yes_iff]
  constructor
  · -- Lemma I.17
    rintro ⟨x, hx⟩
    refine ⟨X.codedE x, ?_⟩
    have hrpe := (X.exactCover_iff_RPE x).mp hx
    have hpen : X.Pen x = 0 := by
      unfold Pen
      refine Finset.sum_eq_zero (fun k hk => ?_)
      have := Finset.mem_Icc.mp hk
      rw [hrpe k this.1 this.2]; push_cast; ring
    have hm := X.master hn x
    have hr := abs_le.mp (X.abs_Res_le hn x)
    have hle : (X.inst.Φ (X.codedE x) : ℚ) ≤ X.Aconst + X.Gam := by
      rw [hm, hpen]; linarith [hr.2]
    unfold Bnd
    exact Int.le_floor.mpr hle
  · -- Lemma I.18
    rintro ⟨E, hE⟩
    obtain ⟨E0, -, hmin⟩ := Finset.exists_min_image (univ : Finset (Finset (Job X.n)))
      X.inst.Φ ⟨E, mem_univ E⟩
    have hmin' : ∀ F, X.inst.Φ E0 ≤ X.inst.Φ F := fun F => hmin F (mem_univ F)
    have hE0 : X.inst.Φ E0 ≤ X.Bnd := le_trans (hmin' E) hE
    -- Gate 1: the anchor is early
    have hanc : Job.anchor ∈ E0 := by
      by_contra h
      have h1 := X.anchor_gate h
      have h2 : X.Cstar = X.inst.Φ (X.codedE (fun _ => true)) :=
        X.Φ_pre_eq_inst (X.mem_codedE_anchor _)
      have := hmin' (X.codedE (fun _ => true))
      omega
    -- Gate 2: every block is coded
    have hbal : ∀ u, X.bq (X.xE E0) u = X.β u := by
      by_contra h
      push Not at h
      have := X.load_lock hn hanc h
      have := hmin' (X.codedE (fun _ => true))
      omega
    obtain ⟨σ, rfl⟩ := X.coded_of_balanced hn hanc hbal
    -- Gate 3: every prefix equation is exact
    refine ⟨σ, (X.exactCover_iff_RPE σ).mpr ?_⟩
    intro k hk1 hkn
    by_contra hne
    have hk : k ∈ Icc 1 X.n := Finset.mem_Icc.mpr ⟨hk1, hkn⟩
    have hm := X.master hn σ
    have hr := abs_le.mp (X.abs_Res_le hn σ)
    have hB : (X.Bnd : ℚ) ≤ X.Aconst + X.Gam := Int.floor_le _
```

```
    have hΦ : ((X.inst.Φ (X.codedE σ) : ℤ) : ℚ) ≤ X.Bnd := by exact_mod_cast hE0
    have hP := Pen_ge hn hk hne
    have hΔ : ((X.G : ℚ) * X.M) < X.Δ k := by exact_mod_cast Δ_gt hn hk
    have h4 := four_Gam_lt hn
    linarith [hr.1]
```

- **If** (Lemma I.17). Choose the coded set of an exact cover x. Its prefix penalty is 0, so the master identity and $|R| \le \Gamma$ give $\Phi \le A + \Gamma$. Since Φ is an integer, $\Phi \le \lfloor A + \Gamma \rfloor = B$ (`Int.le_floor`).
- **Only if** (Lemma I.18). By `yes_iff` some early set has $\Phi \le B$; take a minimizer E_0 of Φ over all (finitely many) early sets. Gate 1: the anchor is early, otherwise $\Phi(E_0) > C^\star = \Phi(\mathrm{codedE}(+1))$, contradicting minimality. Gate 2: every load is β, otherwise `load_lock` contradicts minimality; so $E_0 = \mathrm{codedE}(\sigma)$. Gate 3: if a prefix equation fails, then $\Phi \ge A + \Delta_k/2 - \Gamma > A + \Gamma \ge B$ by the master identity, `Pen_ge`, `four_Gam_lt` and `Δ_gt`, contradicting $\Phi \le B$. So σ solves RPE, and by Reduction I it is an exact cover.

The empty instance is handled separately; then the instance has only the anchor, and both sides are yes-instances:

```
/-- the degenerate empty instance (`n = 0`) is a yes-instance on both sides -/
theorem reduction_correct_zero (hn : X.n = 0) : X.HasExactCover ↔ X.inst.Yes X.Bnd := by
  …  -- 14 lines of tactic proof

/-- **Correctness of the strong reduction RXC3 → AWET (Theorem I.2, Lemmas I.17–I.18).** -/
theorem reduction_iff : X.HasExactCover ↔ X.inst.Yes X.Bnd := by
  rcases Nat.eq_zero_or_pos X.n with h | h
  · exact X.reduction_correct_zero h
  · exact X.reduction_correct h
```

Difference from the paper The paper's only-if proof starts from an optimal schedule and applies Lemma I.3. The formal proof starts from a minimizer of Φ over all early sets, which exists because there are finitely many, and uses `yes_iff`. Gate 3 needs only $\Delta_{\min}/2 > 2\Gamma$; the extra "$+1$" in (8.33) is not used.

3.13 Step 13 — Polynomial and unary bounds (Lemma I.19)

Lemma I.19 Every integer of the constructed instance is bounded by N^{650}, $N = n + 1$.

```
theorem H_le : X.H ≤ X.NN ^ 254 := by
  …  -- 8 lines of tactic proof

theorem Cstar_le : X.Cstar ≤ X.NN ^ 387 := by
  …  -- 30 lines of tactic proof

theorem wL_anchor_le : X.H * X.sa ≤ X.NN ^ 642 :=
  …  -- 1-line term proof

theorem abs_Bnd_le : |X.Bnd| ≤ X.NN ^ 391 := by
```

```
  … -- 27 lines of tactic proof

/-- **Lemma I.19 (polynomial and unary bounds).** The constructed instance has `8n + 1` jobs and
every integer it contains — processing times, earliness and tardiness weights, the due date
and the decision bound — is at most `(n+1)^650` in absolute value. -/
theorem size_bound (X : RXC3) (hn : 0 < X.n) :
    Fintype.card (Job X.n) = 8 * X.n + 1 ∧
    (∀ j, 0 < X.inst.p j ∧ X.inst.p j ≤ ((X.n : ℤ) + 1) ^ 650) ∧
    (∀ j, 0 < X.inst.wE j ∧ X.inst.wE j ≤ ((X.n : ℤ) + 1) ^ 650) ∧
    (∀ j, 0 < X.inst.wL j ∧ X.inst.wL j ≤ ((X.n : ℤ) + 1) ^ 650) ∧
    X.inst.d ≤ ((X.n : ℤ) + 1) ^ 650 ∧
    |X.Bnd| ≤ ((X.n : ℤ) + 1) ^ 650 := by
  … -- 22 lines of tactic proof
```

How the proof goes. The bounds are propagated along the dependency chain of the construction, exactly as in the paper's Steps 1–5, with a handful of helper lemmas: products add exponents (`mul_le_pw`), sums over the $8n \le N^3$ state jobs or the $n \le N$ blocks add 3 or 1 to the exponent (`sum_SJ_le`, `sum_blocks_le`), and constants $c \le 4^k$ are absorbed as N^k because $N \ge 4$ (`mul_pow_le`). Rational quantities (a^0, $a^\star$, e_v, J_{uv}, C_0, A, Γ) are bounded in $\mathbb{Q}$ and transferred to the floor B. Table 2 compares the exponents.

Quantity	Lean	Lean	Paper	Quantity	Lean	Lean	Paper
p_i (state)	`pS_le`	51	51	U	`U_le`	253	253
β_v	`β_le`	52	52	$H = p_a$	`H_le`	254	254
λ_v	`lam_le'`	21	21	s_i	`sS_le`	277	276
L_v, K_{uv}	`K_le'`	22	21	w_i^L (state)	`wLS_le`	328	327
f_v	`f_le`	75	74	w_a^E	`wEJ_anchor_le`	331	330
r_i, ρ, Δ_k	`rS_le`	76	75	$C^\star$	`Cstar_le`	387	385
w_i^E (state)	`wES_le`	127	126	s_a	`sa_le`	388	386
α_{ij}	`α_le`	178	177	w_a^L	`wL_anchor_le`	642	640
$\lvert a_v^0\rvert$	`abs_a0_le`	182	181	d	`d_le`	258	255
$\lvert a_v^\star\rvert$	`abs_astar_le`	79	78	$\lvert C_0\rvert$	`abs_C0_le`	388	385
D_v (upper)	`Dv_le'`	44	43	$\lvert A\rvert$	`abs_Aconst_le`	389	386
$\lvert\eta_v\rvert$	`abs_eta_le`	144	144	$\lvert B\rvert$	`abs_Bnd_le`	391	387

Table 2: Exponents e of the bounds $\le N^e$ proved in Lean, next to the paper's exponents in the proof of Lemma I.19 (strict bounds $< N^e$). The Lean bookkeeping absorbs constants more generously, so a few exponents are 1–4 larger; all are far below 650.

As a point of reference, in the paper's worked example ($n = 6$) and in random instances with $n \le 9$, the largest integer of the constructed instance is about N^{308} (the bound B), and the anchor's tardiness weight is about N^{185}; the exponent 650 is very generous.

3.14 Step 14 — The worked examples of Appendix B

To confirm that the formal construction is the paper's construction, Section 16 of the file evaluates it on the two worked examples of Appendix B with the kernel (`decide`; no `native_decide`):

```
/-- The "yes" instance of Appendix B.1 (elements `e₁,…,e₆` are numbered `0,…,5`). -/
def exYes : RXC3 where
  n := 6
  S := ![![0, 1, 2], ![0, 1, 4], ![1, 3, 5], ![0, 2, 5], ![3, 4, 5], ![2, 3, 4]]
  mono := by decide
  reg := by decide

/-- The "no" instance of Appendix B.2 (`e₃` and `e₄` exchanged between `S₁` and `S₃`). -/
def exNo : RXC3 where
  n := 6
  S := ![![0, 1, 3], ![0, 1, 4], ![1, 2, 5], ![0, 2, 5], ![3, 4, 5], ![2, 3, 4]]
  mono := by decide
  reg := by decide

/-- block `v = 1` of the example (0-based index `0`) -/
def blk1 : Fin exYes.n := ⟨0, by decide⟩

/-- Table 10, first row: `P₁₁` and `β₁`. -/
theorem table10_P11 : exYes.P blk1 1 = 17984650808482467120744482550182601845953355 := by decide
theorem table10_beta1 : exYes.β blk1 = 71938602660917350001165411484001244853761 24 := by decide

/-- Table 13: the first main gap `Δ₁`, the narrowest gap `Δ₄` and the total band width `W`. -/
theorem table13_gap1 : exYes.Δ 1 = 10160436243621332561918604705480016102804028711418 13 := by
  decide
theorem table13_gap4 : exYes.Δ 4 = 10160435056634393973065140045276168363741616731426 06 := by
  decide
theorem table13_W : exYes.W = 11222421986452480676091195490797256354872208 72 := by decide

/-- B.1: `{S₁, S₅}` is an exact cover ... -/
theorem exYes_cover : exYes.IsExactCover ![true, false, false, false, true, false] := by
  unfold RXC3.IsExactCover; decide

/-- ... hence the constructed 49-job AWET instance has a schedule of cost at most `B`. -/
theorem exYes_yes : exYes.inst.Yes exYes.Bnd := exYes.reduction_iff.mp ⟨_, exYes_cover⟩

/-- B.2: the modified instance has no exact cover (all 64 selections fail) ... -/
theorem exNo_noCover : ¬ exNo.HasExactCover := by
  unfold RXC3.HasExactCover RXC3.IsExactCover; decide

/-- ... hence its constructed AWET instance has **no** schedule (with arbitrary real start
times) of cost at most `B`. -/
theorem exNo_no : ¬ exNo.inst.Yes exNo.Bnd := fun h => exNo_noCover (exNo.reduction_iff.mpr h)
```

The five numerical theorems reproduce, digit for digit, the paper's Table 10 (first row) and Table 13 (the first main gap, the narrowest main gap $\Delta_{\min} = \Delta_4$, and the total band width W). The last two theorems use the main theorem in both directions: the 49-job instance built from the yes-example has a schedule of cost at most B, and the instance built from the no-example has *no* schedule of cost at most B, among all schedules with arbitrary real start times. This is a statement that no finite computation over schedules could establish.

4 Remarks on the paper

No error was found. All statements on the path to Theorem I.2 hold as stated: Lemma I.3 and Proposition I.4 (in the form $\mathrm{OPT} = \min_E \Phi(E)$), Lemmas I.7–I.9 and I.11–I.19, and the equations used in between. The construction of Section 8.1 is well defined and works exactly as described. The remarks below are minor; none of them affects the result.

1. **Small instances.** Section 8 assumes $n \geq 3$, "since smaller instances can be decided directly". For RXC3 this assumption is automatic: a nonempty instance contains a set of three distinct elements, so $n \geq 3$ (`n_ge_three`). The empty instance is a yes-instance, and the construction still produces a valid one-job instance with $B = 0$ that is a yes-instance, so no case distinction is needed in the reduction (`reduction_correct_zero`).
2. **Proposition I.4 for $E = \emptyset$.** The proposition is stated for nonempty E, but the all-tardy assignment is also realized by a feasible schedule (start at d) of cost $\Phi(\emptyset)$. Allowing $E = \emptyset$ is harmless and slightly simplifies the argument.
3. **Choices left open.** The paper allows any coded assignment as the benchmark for $C^\star$ (8.25) and either nearest integer in (8.17). The formalization fixes $\sigma \equiv +1$ and round $x = \lfloor x + \frac{1}{2} \rfloor$; the proofs use only the properties the paper states. The constant C_0 is described as "the computable constant coefficient" of (8.26); the formalization computes it from the benchmark and proves that this is indeed the constant coefficient (`coded_objective`). The paper's bound on C_0 by averaging over all sign vectors is replaced by a direct bound.
4. **The "+1" in (8.33).** Gate 3 only needs $\Delta_{\min}/2 > 2\Gamma$: then $\Phi \geq A + \Delta_{\min}/2 - \Gamma > A + \Gamma \geq \lfloor A + \Gamma \rfloor = B$. The additional unit "to accommodate the integer floor" is not needed (but does no harm).
5. **Exponent bookkeeping in Lemma I.19.** The formal bounds differ from the paper's intermediate exponents by 1–4 in places (Table 2) because constants are absorbed more coarsely. We did not try to reproduce the paper's sharper intermediate exponents, which look plausible; the paper's final claim, that every integer is at most N^{650}, is confirmed.
6. **Repeated sets.** Definition I.5 speaks of a family of sets; the formal reduction is correct even if some sets coincide, so it covers a slightly larger class of inputs.
7. **Proof routes.** Three arguments are replaced by more elementary equivalents in the formalization: the schedule surgery in Lemma I.3 (Step 2), the profile integral in (8.10) (Step 7), and the optimality formulation of Lemma I.15 (Step 9). The paper's arguments for these steps are correct as informal proofs; the replacements were chosen because they are shorter to formalize.

What remains outside the formal proof. (i) The NP-completeness of RXC3 (Gonzalez, 1985), cited by the paper. (ii) The polynomial running time of the transformation (Lemma I.19, Step 6): the Lean definitions are explicit formulas over $O(n^2)$ terms whose numbers have $O(\log n)$ bits, and `size_bound` bounds every output number by $(n+1)^{650}$, so the unary size is polynomial as well; but running time itself is not a formal statement in the file. (iii) The standard meta-argument (Garey and Johnson) that (a)–(c) of Section 1 imply strong NP-hardness. All three are textbook facts that the paper also takes as given.

5 Reproducing the check

- **Toolchain.** Lean 4 v4.35.0 (commit `eb6cafb7`) and Mathlib at tag `v4.35.0` (commit `56a00585`). Any Lake project that depends on this Mathlib version will do; the file needs nothing else.
- **Command.** `lake env lean AWET_PartI.lean`. On the machine used for this report the file compiles in about one minute.
- **Expected output.** No errors and no warnings; exactly the three lines

```
#print axioms AWET.AWET_strong_reduction
#print axioms AWET.Inst.yes_iff
#print axioms AWET.Examples.exNo_no
```

  each followed by `depends on axioms: [propext, Classical.choice, Quot.sound]`.
- **Optional kernel replay.** Compile to an `.olean` (`lean -o AWET_PartI.olean AWET_PartI.lean`) and run `leanchecker AWET_PartI` with the same `LEAN_PATH`; it exits with code 0 (about 10 seconds).
- **File fingerprint.** SHA-256 of the verified file:
  `87c9119b2c3b35a716ddab3de0e537cfd5e0f9a6791364f1f15243a6664a0419`.

Numerical cross-check before formalization. Independently of Lean, the construction was first reimplemented in Python with exact rational arithmetic and run on the paper's yes-example and on random RXC3 instances with $n \in \{3, 6, 9\}$. For every instance it checked Lemma I.11 by enumerating all 2^8 sign vectors of every block, the claims of Lemmas I.12 and I.14, the bounds on $J_{uv} - J_{uv}^{\text{main}}$, e_v and Γ, inequality (8.33), the anchor gate, and the master identity (8.31) together with $|R(\sigma)| \le \Gamma$ and the correct yes/no outcome for *every* coded state $\sigma \in \{\pm 1\}^n$. All checks passed.

A Map from the paper to the Lean file

Paper	Lean	Status
Definition I.1 (AWET)	`Inst`, `Inst.d`, `Inst.Sched`, `Inst.cost`, `Inst.Yes`	formalized
Theorem I.2	`AWET_strong_reduction`	✓ (formal core)
Lemma I.3, (6.1)	`Sched.pack`, `Φ_le_ordCost`, `exists_Φ_le_cost`	✓ (other proof)
Proposition I.4, (SO)	`Inst.Φ`, `canC`, `exists_sched_cost_eq`, `yes_iff`	✓
Definitions I.5, I.6	`RXC3`, `IsExactCover`, `dpre`, `Cpre`, `RPE`	formalized
Lemma I.7	`Cpre_succ`	✓
Lemmas I.8, I.9	`exactCover_iff_RPE`	✓
Corollary I.10	—	not needed
Section 8.1 (Tables 1–2, (8.1)–(8.2), (8.5), (8.14)–(8.22), (8.25), (8.29), (8.32), (8.34))	definitions in Section 3 of the file; `RXC3.inst`, `RXC3.Bnd`	formalized
Lemma I.11, (8.4)	`digit_decode`, `pair_decode`, `two_states`, `bq_coded`	✓
Lemma I.12	`β_bounds`, `f_sub_bounds`, `f_strict`	✓ (bounds used)
Lemma I.13	`pair_compress`, `J4_compress`	✓
Lemma I.14, (8.8)	`scale_key`, `phys_sep`, `eff_sep`, `eff_gap`, `eff_width`, `gap_gt`, `W_le`	✓

Paper	Lean	Status
(8.9)–(8.12)	`layer_decomp`, `gp_atom`, `Mab_eq_dpre`, `gapSum_eq`, `bandSum_bounds`, `J4_decomp`	✓ (discrete form)
(8.13)	`sum_dpre`, inside `master`	✓
Lemma I.15, (8.23), (8.24)	`Φ_anchor_early`, `phiQ_decomp`, `quad_prefix`, `quad_ge_two`, `R0_bounds`, `load_lock`	✓ (stronger form)
Anchor tardiness (8.25)	`anchor_gate`, `Φ_pre_eq_inst`	✓
(8.26)	`a0_eq`, `R0_coded`, `coded_objective`	✓
Lemma I.16, (8.27)–(8.31)	`Jmain`, `Pen`, `Res`, `master`	✓
(8.19), (8.32)	`abs_ev_le`, `Jc_sub_Jmain`, `abs_Res_le`	✓
(8.33)	`four_Gam_lt`, `Δ_gt`	✓ (without "+1")
Lemma I.17	`reduction_correct` (⇒)	✓
Lemma I.18	`reduction_correct` (⇐), `coded_of_balanced`, `Pen_ge`	✓
Lemma I.19	`size_bound` and the bounds of Table 2	✓ (numbers)
Appendix B.1, B.2	`AWET.Examples`	✓ (selected values)

B Layout of the Lean file

Lines	Thms	Defs	Section of the file
1–58	–	–	Header comment: claim, what is proved, how to check, map of the file
59–78	–	–	Imports (Mathlib only)
79–87	–	–	Options and namespace set-up
88–132	3	0	Generic pair-sum lemmas
133–775	30	18	1. The AWET problem (with the canonical schedule, Prop. I.4, and the lower bound, Lemma I.3)
776–804	0	6	2. Restricted Exact Cover by 3-Sets (RXC3) and Regular Prefix Exactness (RPE)
805–1179	40	55	3. The construction of Section 8 (with positivity of all constructed data)
1180–1515	34	1	4. Basic arithmetic of the construction (Lemma I.11)
1516–1975	35	3	5. Kernel field (Lemma I.12) and scale separation (Lemma I.14)
1976–2523	41	9	6. Effective bands, main gaps and the profile decomposition of J_{uv}
2524–2604	6	2	7. The canonical objective on anchor-early sets
2605–2880	17	8	8. Quadratic bookkeeping over state jobs (in Q)
2881–3243	20	4	9. The global load lock (Lemma I.15) and the anchor gate
3244–3543	9	5	10. Expansion on the coded state space and the master identity (Lemma I.16)
3544–3649	7	0	11. Reduction I: RXC3 ⇔ RPE (Lemmas I.7–I.9)
3650–4059	19	3	12. Master identity (Lemma I.16), residual bound and decision gap
4060–4183	4	0	13. Main theorem
4184–4918	61	0	14. Polynomial size bounds (Lemma I.19)
4919–4946	1	0	15. Summary: Part I of Hall–Kellerer–Song, machine-checked
4947–5002	9	3	16. Sanity check against the paper's worked examples (Appendix B)
total: 5002 lines	336	117	